

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Liquid Types para la verificación de *smart contracts* con diccionarios

Tesis de Licenciatura en Ciencias de la Computación

Tomás Jarabroviski

Director: Dr. Hernán Melgratti

Buenos Aires, 2026

Índice general

1.. Soporte de mappings 1
1.1. Semántica de un mapping 2
1.2. Especificación en Liquid Haskell 2
1.3. Traducción de las operaciones 4

1. SOPORTE DE MAPPINGS

Los *mappings* (diccionarios) son una de las estructuras de datos centrales en los contratos que manejan fondos: los saldos de cada cuenta en un token o los montos depositados por cada usuario en un banco se guardan en un *mapping* de direcciones en enteros, `mapping (address => uint)`. En [Con25], donde se presentó la traducción de Solidity a Haskell que este trabajo extiende, la posibilidad de representar *mappings* se estudió solo de forma preliminar, mediante una exploración manual que no llegó a incorporarse a la traducción automática. En este capítulo desarrollamos ese soporte de manera completa.

El problema tiene dos niveles. El primero es de especificación: definir en Haskell una representación de los *mappings* junto con tipos refinados que describan con precisión el efecto de sus operaciones, es decir, qué valor se lee después de una escritura, qué entradas no cambian y cómo evoluciona la suma de los valores. El segundo es de traducción: hacer que el traductor exprese cada operación de Solidity sobre un *mapping* en términos de esa representación, de modo que Liquid Haskell pueda verificar automáticamente las funciones del contrato que la usan. Los tipos refinados resultan adecuados para ambos niveles, porque permiten expresar los invariantes de la estructura y las garantías de cada operación como parte de los tipos, y delegar su verificación a un *solver* SMT [VSJ+14b].

A lo largo del capítulo utilizaremos como ejemplo el token mínimo del Listado 1.1. El constructor asigna todo el suministro inicial a la cuenta que despliega el contrato; `transfer` mueve `amount` unidades desde el saldo de quien invoca la función (`msg.sender`) hacia el de `to`, y `burn` destruye `amount` unidades del saldo de quien la invoca. Ambas funciones exigen, mediante `require`, que el saldo del remitente sea suficiente.

```
contract Token {
    mapping(address => uint) balances;

    constructor(uint initialSupply) {
        balances[msg.sender] = initialSupply;
    }

    function transfer(address to, uint amount) public {
        require(balances[msg.sender] >= amount);
        balances[msg.sender] -= amount;
        balances[to] += amount;
    }

    function burn(uint amount) public {
        require(balances[msg.sender] >= amount);
        balances[msg.sender] -= amount;
    }
}
```

Listado 1.1: Token mínimo con un *mapping* de saldos.

1.1. Semántica de un mapping

Un *mapping* de Solidity se comporta como una función total de claves en valores. La traducción debe preservar las siguientes propiedades:

Valor por defecto. Toda clave tiene un valor asociado: si nunca fue escrita, su valor es el valor por defecto del tipo, que para `uint` es 0. En el ejemplo, `balances[to]` vale 0 para cualquier cuenta que nunca recibió tokens. En consecuencia, un contrato no puede distinguir una clave que nunca fue escrita de una cuyo valor almacenado es 0, y `delete m[k]` equivale a `m[k] = 0`.

Escrituras locales. Las sentencias `m[k] = e`, `m[k] += e` y `m[k] -= e` modifican únicamente la entrada `k`; el resto del *mapping* no cambia.

Valores no negativos. Los valores de tipo `uint` nunca son negativos. En este capítulo los enteros se interpretan como enteros matemáticos sin desbordamiento[1], por lo que la única restricción es que todo valor almacenado sea mayor o igual a 0.

La última propiedad es la que convierte a los *mappings* en una fuente de errores. En `burn`, el `require` garantiza que el saldo resultante no sea negativo; si se omitiera, cualquier cuenta podría destruir tokens que no tiene. Queremos que la traducción transforme esta propiedad en una obligación de prueba, de modo que Liquid Haskell rechace la versión sin `require`.

Una alternativa natural sería representar el *mapping* como una función de Haskell `k -> Int`. Sin embargo, además de leer y escribir claves individuales, nos interesa poder razonar sobre la suma de todos los valores (por ejemplo, el total de tokens en circulación), lo cual requiere una representación finita y explícita de las entradas.

1.2. Especificación en Liquid Haskell

Las definiciones del Listado 1.2 forman parte de una biblioteca que se incluye en cada módulo generado que utiliza algún *mapping*.

```
{-@ type Map k v = {m : [(k, v)] | isUniq m} @-}

{-@ reflect sumBids @-}
sumBids :: Map k Int -> Int
sumBids []            = 0
sumBids ((_, x) : xs) = x + sumBids xs

{-@ data MapWithBalance k = MB { mMap :: Map k {v:Int | InRange v}
                               , mBal :: {v:Int | v >= 0} } @-}

{-@ type BalancedMap k =
      {m : MapWithBalance k | sumBids (mMap m) == mBal m} @-}
```

Listado 1.2: Representación de un *mapping* (definiciones principales).

Las definiciones se leen de la siguiente manera:

[1] El tratamiento del desbordamiento aritmético de `uint` se presenta en un capítulo posterior.

- `Map k v` es una lista de pares clave–valor. El predicado `isUniq` exige que ninguna clave aparezca dos veces, de modo que cada clave tiene a lo sumo un valor asociado.
- `sumBids` calcula la suma de los valores del mapa. La anotación `reflect` permite usar esta función dentro de los tipos refinados.
- `MapWithBalance k` acompaña al mapa con un campo `mBal` que guarda la suma de sus valores. Los valores del mapa deben cumplir el predicado `InRange`, que en este capítulo expresa que son no negativos.
- `BalancedMap k` es el tipo que efectivamente usa la traducción: un `MapWithBalance` cuyo campo `mBal` coincide con la suma real de sus valores. Este invariante es parte del tipo, por lo que toda función que devuelva un `BalancedMap` debe preservarlo, y Liquid Haskell lo verifica.

La lista de pares no aparece en el código generado para las funciones del contrato: este solo accede al mapa a través de las tres operaciones de la Tabla 1.1.

```
empWB          :: {m:BalancedMap k | mBal m == 0}
currentBalance :: m:BalancedMap k -> key:k -> {v:Int | InRange v}
setWB          :: m:BalancedMap k -> key:k -> val:{Int | InRange val}
               -> {n:BalancedMap k |
                     currentBalance n key == val &&
                     mBal n == mBal m - currentBalance m key + val}
```

Tab. 1.1: Interfaz de `BalancedMap` (tipos simplificados).

- `empWB` es el mapa vacío, cuya suma es 0.
- `currentBalance m key` devuelve el valor asociado a `key`, o 0 si la clave no está en el mapa. Es la operación que implementa la semántica de valor por defecto: internamente se distingue una clave ausente de una con valor 0, pero `currentBalance` devuelve lo mismo en ambos casos, tal como ocurre en Solidity.
- `setWB m key val` asocia `val` a `key`, ya sea agregando la clave o reemplazando su valor anterior, y actualiza la suma: se resta el valor previo y se suma el nuevo. Su argumento `val` debe cumplir `InRange`; esta precondición es la que permite detectar saldos negativos.

Los tipos de la Tabla 1.1 resumen las especificaciones de la biblioteca, que están expresadas en términos de la pertenencia de la clave al mapa y de la función `get`. En conjunto, la biblioteca establece las siguientes garantías, para todo mapa `m`, claves `k` y `k'`, y valor `v` que cumpla `InRange`:

Lectura por defecto. Si `k` no está en el mapa, `currentBalance m k == 0`; en particular, toda lectura sobre `empWB` da 0.

Lectura tras escritura. `currentBalance (setWB m k v) k == v`.

Marco. Si `k' /= k`, entonces `currentBalance (setWB m k v) k' == currentBalance m k'`: escribir una clave no modifica las demás.

Suma. `mBal (setWB m k v) == mBal m - currentBalance m k + v`, y el invariante `sumBids (mMap m) == mBal m` se preserva.

Cota. `currentBalance m k <= mBal m`: ningún valor supera la suma.

Estas garantías están verificadas por Liquid Haskell dentro de la propia biblioteca, mediante lemas auxiliares sobre listas y conjuntos; solo dos propiedades elementales de la teoría de conjuntos se asumen sin prueba. No es necesario conocer esos lemas para seguir la traducción.

1.3. Traducción de las operaciones

En la traducción base, el estado del contrato se representa con un registro de Haskell, cada función de Solidity se traduce en una función que recibe el estado actual (`st`) y sus parámetros y devuelve el nuevo estado, y una secuencia de sentencias se traduce como una cadena de estados intermedios `st1`, `st2`, ... ligados con `let`. Un `require` se traduce de dos formas: como precondición en el tipo refinado de la función y como una sentencia que revierte (`revert`) si la condición no se cumple.

Sobre ese esquema, la Tabla 1.2 muestra cómo se traducen las operaciones sobre un *mapping* `m` declarado en un contrato `C`. Al igual que el resto de los campos del estado, su nombre lleva como sufijo el nombre del contrato.

Solidity	**Haskell**
`mapping(K => uint) m`	campo `mC :: BalancedMap K`
valor inicial	`empWB`
`m[k]`	`currentBalance (mC st) k`
`m[k] = e`	`setWB (mC st) k e`
`m[k] += e`	`setWB (mC st) k (currentBalance (mC st) k + e)`
`m[k] -= e`	`setWB (mC st) k (currentBalance (mC st) k - e)`

Tab. 1.2: Traducción de las operaciones sobre un *mapping*. En las tres escrituras, el mapa resultante se asigna al campo `mC` del nuevo estado.

A continuación aplicamos la traducción al token del Listado 1.1.

Constructor. El constructor construye el estado inicial completo. El *mapping* parte del mapa vacío y, como el constructor escribe una clave, se le aplica la escritura correspondiente:

```
constructorToken msgctx block initialSupply = TokenState
  { ...
  , balancesToken = setWB empWB (sender msgctx) initialSupply }
```

La obligación que impone `setWB` es que `initialSupply` sea no negativo, lo cual se cumple porque el parámetro tiene tipo `Uint`.

`burn`. La función se traduce de la siguiente manera, donde `bal` y `me` abrevian, respectivamente, `balancesToken` y `sender (msgToken st)`[2]:

[2] En el código generado se usan los nombres completos y aparecen además las aserciones de lemas que describimos al final de la sección.

```
{-@ burn :: {st:TokenState | True}
         -> {amount:Uint | currentBalance (bal st) me >= amount}
         -> {v:TokenState | True} @-}
burn st amount = let
  st1 = if currentBalance (bal st) me >= amount
          then st else revert "error"
  st2 = st1 { bal = setWB (bal st1) me (currentBalance (bal st1) me - amount) }
  in st2
```

Listado 1.3: Traducción de `burn`.

El `require` aparece dos veces: como precondición sobre `amount` y como la sentencia que define `st1`. La escritura `balances[msg.sender] -= amount` se traduce en `st2`. Para aceptar la llamada a `setWB`, Liquid Haskell debe probar que el nuevo valor, `currentBalance (bal st1) me - amount`, es no negativo. Esto se deduce de la precondición, ya que `st1` es igual a `st`.

Si se elimina el `require` de `burn`, la precondición desaparece y esa obligación ya no puede probarse: Liquid Haskell rechaza la llamada a `setWB` y la herramienta reporta el contrato como inseguro. El error de programación (permitir que una cuenta destruya tokens que no tiene) se manifiesta como un error de tipos en el código traducido.

`transfer`. La función realiza dos escrituras, que se traducen en dos estados consecutivos (usando las mismas abreviaturas):

```
  st2 = st1 { bal = setWB (bal st1) me (currentBalance (bal st1) me - amount) }
  st3 = st2 { bal = setWB (bal st2) to (currentBalance (bal st2) to + amount) }
```

La primera escritura genera la misma obligación que en `burn`, que se cumple gracias al `require`. La segunda obligación es inmediata: la suma de dos valores no negativos es no negativa. Notemos que la segunda escritura lee el saldo de `to` a partir de `st2`, es decir, después de la primera escritura. De esta forma la traducción es correcta incluso cuando `to` y `msg.sender` son la misma cuenta, sin necesidad de tratar ese caso por separado.

Supongamos ahora que el programador se equivoca de clave y escribe `require(balances[to] >= amount)`. La precondición pasa a hablar del saldo de `to`, que no dice nada sobre el saldo de `msg.sender`; la obligación de la primera escritura deja de poder probarse, y la herramienta reporta nuevamente el contrato como inseguro. En cambio, la versión original del Listado 1.1 es verificada por Liquid Haskell.

Lemas en el código generado. Cada escritura sobre un *mapping* aparece en el código generado acompañada de aserciones de la forma

```
assert (lemma_currentBalance_get (bal st1) me) $ ...
```

La función `assert :: a -> b -> b` ignora su primer argumento y devuelve el segundo; su único propósito es incorporar al contexto de Liquid Haskell el enunciado del lema, que en este caso relaciona `currentBalance` con la función `get` en términos de la cual están especificadas las operaciones de la biblioteca. El traductor inserta estos lemas de forma sistemática en cada escritura, para que el razonamiento no dependa de que Liquid Haskell

descubra por sí mismo esa relación. Con el mismo mecanismo se insertan las garantías de marco y de cota en las situaciones que lo requieren, que se presentan en capítulos posteriores.

El soporte presentado cubre los *mappings* con valores de tipo `uint` declarados como variables de estado. Quedan fuera los *mappings* con valores de otros tipos (como `bool` o estructuras), los *mappings* usados como parámetros o variables locales, los *mappings* anidados (`mapping(address => mapping(address => uint))`, usados por ejemplo para los permisos de gasto entre cuentas de un token) y la sentencia `delete`; las sentencias que involucran estos dos últimos casos se reportan como no traducidas. Por otro lado, el campo `mBal` y la garantía de cota, que en este capítulo solo se utilizaron para mantener el invariante del mapa, son las piezas que permitirán en el capítulo siguiente relacionar los saldos registrados en un *mapping* con el ether que efectivamente posee el contrato.

Bibliografía

[Con25] Gonzalo Consoli. Liquid types para la verificación de smart contracts. Tesis de licenciatura, Departamento de Computación, Facultad de Ciencias Exactas y Naturales, Universidad de Buenos Aires, septiembre 2025. Director: Dr. Hernán Claudio Melgratti.

[HVH19] Martin A. T. Handley, Niki Vazou, and Graham Hutton. Liquidate your assets: Reasoning about resource usage in Liquid Haskell. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–27, 2019.

[LPR+20] Yiyun Liu, James Parker, Patrick Redmond, Lindsey Kuper, Michael Hicks, and Niki Vazou. Verifying replicated data types with typeclass refinements in Liquid Haskell. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30, 2020.

[NSC+22] Yuki Nishida, Hiromasa Saito, Ran Chen, Akira Kawata, Jun Furuse, Kohei Suenaga, and Atsushi Igarashi. Helmholtz: A verifier for Tezos smart contracts based on refinement types. *New Generation Computing*, 40(2):507–540, 2022.

[RKJ08] Patrick M. Rondon, Ming Kawaguci, and Ranjit Jhala. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 159–169, 2008.

[Sza97] Nick Szabo. Formalizing and securing relationships on public networks. *First Monday*, 1997.

[VSJ14a] Niki Vazou, Eric L. Seidel, and Ranjit Jhala. LiquidHaskell: Experience with refinement types in the real world. In *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, pages 39–51, 2014.

[VSJ+14b] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, pages 269–282, 2014.